



Write

I Built a Full DevOps CI/CD Pipeline from Scratch — Here's Everything That Went Wrong

Updated March 14, 2026 • 20 min read • View as Markdown

```
pipeline {
  agent any
  stages {
    ✓ Clone Code 0.91s
    ✓ Build Docker Image 45s
    ✓ Deploy with Docker Compose 3m 48s
    ✓ Deployment Status 24s
  }
}
```

```
$ docker ps
flask-app Up 2 minutes 0.0.0.0:5000->5000/tcp
mysql Up 3 minutes (healthy) 3306/tcp

$ terraform apply
aws_instance.flask_server: Creating...
Apply complete! Resources: 2 added.
ec2_public_ip = "3.110.85.1"
```

Error: disk full - 99.8% of 6.71GB used
Fixed: upgraded volume 8GB -> 20GB via Terraform
+ growpart + resize2fs

I Built a Full DevOps CI/CD Pipeline from Scratch

Here's Everything That Went Wrong



8 real errors.
8 real fixes.
1 working pipeline.

Shlok Bam
aws-ec2-terraform-dev

#DevOps

Shlok Bam

A honest, detailed walkthrough of building a Flask + Docker + Jenkins + Terraform + AWS project — including every error, every fix, and every "why is this not working" moment.

Before We Start — Why I Built This

I'm learning DevOps. And like most people learning DevOps, I was drowning in theory. I knew what Docker was. I could explain CI/CD in an interview. But I hadn't actually built a full pipeline from scratch.

So I decided to stop watching tutorials and just build something real.

The goal was simple — take a Flask web app, containerize it with Docker, provision cloud infrastructure with Terraform, and set up Jenkins to automatically deploy every time I push code to GitHub.

Simple in theory. Absolutely chaotic in practice.

This is the full story — every step, every error, every fix, and every "oh that's why" moment. If you're learning DevOps and want something real to build, follow along.

What I Built

A simple **Task Manager web app** — you can add tasks, mark them done, delete them. Nothing fancy. The point wasn't the app. The point was the pipeline around it.

Here's what the full setup looks like:



Every time I push code → webhook triggers Jenkins → Jenkins builds and deploys automatically → changes are live in minutes. No manual steps.

Tech stack:

What	Tool
Web App	Python Flask
Database	MySQL 8.0
Containerization	Docker + Docker Compose
Infrastructure	Terraform
CI/CD	Jenkins
Cloud	AWS EC2 (Mumbai region)
Version Control	GitHub

Let's build it step by step.

Phase 1 — The Flask App

First things first — I needed an actual app to deploy. I built a simple Task Manager with Flask and MySQL.

The app has 5 routes:

```
@app.route("/")          # show all tasks
@app.route("/add")         # add a new task
@app.route("/toggle<id>") # mark done/undone
```

```
@app.route("/delete/<id>") # delete a task
@app.route("/health")      # health check for Docker
```

That `/health` route matters — Docker uses it to know when the container is actually ready to accept connections. More on that later.

One thing I was careful about — Flask connects to MySQL using **environment variables**, not hardcoded credentials:

```
def get_db_connection():
    conn = mysql.connector.connect(
        host=os.environ.get("MYSQL_HOST", "localhost"),
        user=os.environ.get("MYSQL_USER", "root"),
        password=os.environ.get("MYSQL_PASSWORD", "root"),
        database=os.environ.get("MYSQL_DB", "devops")
    )
    return conn
```

These values get passed in by Docker Compose later. This is the right way to handle config — keep it out of your code.

The app also auto-creates the database table on startup:

```
def init_db():
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS tasks (
            id INT AUTO_INCREMENT PRIMARY KEY,
            title VARCHAR(255) NOT NULL,
            done BOOLEAN DEFAULT FALSE,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    """)
    conn.commit()
```

No manual SQL setup needed. The table just appears on first run.

Phase 2 — Dockerizing the App

Writing the Dockerfile

The Dockerfile defines how to build the Flask app into a Docker image:

```
FROM python:3.9-slim

WORKDIR /app

RUN apt-get update && apt-get install -y gcc default-libmysqlclient-dev && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

Let me explain what each part actually does, because I spent time understanding this:

- `python:3.9-slim` — lightweight Python base image. The full Python image is 900MB+. Slim is ~130MB. Smaller image = faster builds and pulls.
- `gcc` and `default-libmysqlclient-dev` — the `mysql-connector-python` package needs these to compile. Without them, `pip install` fails with a cryptic error.
- `COPY requirements.txt` before `COPY . .` — this is intentional. Docker caches each layer. If you copy requirements.txt first and install dependencies, Docker only reinstalls packages when requirements.txt actually changes — not every time you change your app code. Saves minutes on every build.

Writing Docker Compose

One container for Flask, one for MySQL. Docker Compose manages both:

```
version: "3.8"

services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: "devops"
      MYSQL_ROOT_PASSWORD: "root"
    ports:
      - "3306:3306"
    volumes:
      - mysql-data:/var/lib/mysql
    networks:
      - two-tier
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uro
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 60s

  flask:
    build:
      context: .
    ports:
      - "5000:5000"
    environment:
      - MYSQL_HOST=mysql
      - MYSQL_USER=root
      - MYSQL_PASSWORD=root
      - MYSQL_DB=devops
    networks:
      - two-tier
    depends_on:
      mysql:
        condition: service_healthy

volumes:
  mysql-data:

networks:
  two-tier:
```

Three things here that actually matter:

1. Docker networking — notice `MYSQL_HOST=mysql`. Flask connects to MySQL using the container name `mysql` as the hostname — not `localhost`. This is how Docker networking works. Containers on the same network can reach each other by their service name. This confused me initially until it clicked.

2. Healthcheck + depends_on — `depends_on: condition: service_healthy` means Flask only starts after MySQL passes its healthcheck. Without this,

Flask starts while MySQL is still initializing, can't connect, and crashes. The healthcheck pings MySQL every 10 seconds. Only when it gets a successful response does Flask start.

3. Named volume — `mysql-data:/var/lib/mysql` stores MySQL data in a named volume, not inside the container. This means your data survives container restarts and even redeployments. Without this, every `docker compose down` would wipe all your data.

First Problem — Port 3306 Already in Use

I ran `docker compose up -d --build` and got this:

```
Error response from daemon: ports are not available: exposing port
0.0.0.0:3306 -> 127.0.0.1:0: listen tcp 0.0.0.0:3306: bind: address
```

My Mac had MySQL installed locally and already using port 3306.

Fix: Changed the port mapping in `docker-compose.yml` from `3306:3306` to `3307:3306`. This means my Mac uses port 3307 externally, but inside Docker's network containers still communicate on 3306. Flask was unaffected because Flask talks to MySQL *inside* the Docker network, not through the host port.

```
docker compose down
docker compose up -d --build
docker ps
```

```
❯ [..venv] shlok@shlok-MacBook-Air-2: terraform % docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NA
f183183244c48 flask-todo-app-flask               "python app.py"         17 hours ago   Up 17 hours   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   f1
ask-app        8672a49c1ce7 mysql:8.0                         "docker-entrypoint.s..." 17 hours ago   Up 17 hours (healthy)  0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp   my
sql
```

Verifying MySQL Actually Works

This is something I'd recommend everyone do — don't just trust the UI. Connect directly to MySQL and verify:

```
docker exec -it mysql mysql -uroot -proot devops
```

```
SHOW TABLES;
SELECT * FROM tasks;
```

I could see my tasks in the database. `done = 1` for completed tasks, `done = 0` for pending. The auto-increment IDs had gaps (1, 2, 4) because I'd deleted task 3 — completely normal MySQL behaviour.

Phase 1 done. Flask app + MySQL running locally in Docker, data persisting correctly.

Phase 3 — AWS Infrastructure with Terraform

Now I needed to get this running on AWS. But instead of clicking through the AWS console, I used Terraform to define the infrastructure as code.

What is Terraform and Why Use It?

Terraform is a tool that lets you describe your cloud infrastructure in code files. Instead of manually clicking through 10 screens in AWS console to create

an EC2 instance, you write a `.tf` file and run one command. Terraform makes the API calls to AWS and creates everything.

The benefit is repeatability. If I need to recreate my infrastructure, I just run `terraform apply` again. If someone else wants to run this project, they run the same command and get identical infrastructure. No more "it worked on my account" problems.

Step 1 — Create IAM User

First rule of AWS — never use root credentials for programmatic access. I created a dedicated IAM user:

1. AWS Console → IAM → Users → Create User
2. Username: `terraform-user`
3. Attach policy: `AdministratorAccess`
4. Security credentials tab → Create access key → CLI use case
5. Download the CSV — **you only see the secret key once**

Step 2 — Configure AWS CLI

```
brew install awscli
aws configure
```

Entered the access key, secret key, region (`ap-south-1` — Mumbai, closest to me in India), and output format (`json`).

Verified it worked:

```
aws sts get-caller-identity
```

```
{
  "UserId": "AIDAVYL6B7OWOE46SJFVF",
  "Account": "395938234560",
  "Arn": "arn:aws:iam::395938236560:user/terraform-user"
}
```

This command asks AWS "who am I?" — if it returns your account details, credentials are configured correctly. If Terraform can run this command, it can create resources in your account.

Step 3 — The Three Terraform Files

`variables.tf` — stores values that might change:

```
variable "aws_region" {
  default = "ap-south-1"
}

variable "instance_type" {
  default = "t2.micro"
}

variable "key_name" {
  description = "Your EC2 key pair name"
}
```

`key_name` has no default — Terraform will ask for it every time you run `apply`. This is intentional because key pair names are personal to each AWS account.

main.tf — the actual AWS resources:

```
provider "aws" {  
  region = var.aws_region  
}  
  
data "aws_vpc" "default" {  
  default = true  
}  
  
data "aws_subnets" "default" {  
  filter {  
    name = "vpc-id"  
    values = [data.aws_vpc.default.id]  
  }  
}  
  
resource "aws_security_group" "flask_sg" {  
  name = "flask-jenkins-sg"  
  description = "Allow SSH, Jenkins, and Flask"  
  
  ingress {  
    description = "SSH"  
    from_port = 22  
    to_port = 22  
    protocol = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  
  ingress {  
    description = "Jenkins"  
    from_port = 8080  
    to_port = 8080  
    protocol = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  
  ingress {  
    description = "Flask App"  
    from_port = 5000  
    to_port = 5000  
    protocol = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  
  egress {  
    from_port = 0  
    to_port = 0  
    protocol = "-1"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}  
  
resource "aws_instance" "flask_server" {  
  ami = "ami-0f58b397bc5c1f2e8"  
  instance_type = var.instance_type  
  key_name = var.key_name  
  vpc_security_group_ids = [aws_security_group.flask_sg.id]  
  subnet_id = tolist(data.aws_subnets.default.ids)[0]  
  associate_public_ip_address = true  
  
  root_block_device {  
    volume_size = 20  
  }  
  
  tags = {  
    Name = "flask-jenkins-server"  
  }  
}
```

The security group is basically a firewall. Port 22 for SSH, 8080 for Jenkins, 5000 for Flask. Without opening these ports, nothing is reachable from outside the EC2.

`outputs.tf` — prints useful info after apply:

```
output "ec2_public_ip" {  
  value = aws_instance.flask_server.public_ip  
}  
  
output "ssh_command" {  
  value = "ssh -i ~/.ssh/{var.key_name}.pem ubuntu@{aws_instance.  
}
```

This prints your EC2 IP and exact SSH command after Terraform finishes. I love this — no need to go back to AWS console to find the IP.

Step 4 — Create Key Pair

In AWS Console → EC2 → Key Pairs → Create:

- Name: `flask-key`
- Type: RSA, Format: `.pem`
- Download it

Then on my Mac:

```
mv ~/Downloads/flask-key.pem ~/.ssh/  
chmod 400 ~/.ssh/flask-key.pem
```

`chmod 400` makes the key readable only by you. SSH refuses to use keys with loose permissions — you'll get "WARNING: UNPROTECTED PRIVATE KEY FILE" and the connection gets rejected.

Step 5 — Terraform Init, Plan, Apply

```
cd terraform  
terraform init
```

This downloads the AWS provider plugin. You'll see a `.terraform` folder appear. The `.terraform.lock.hcl` file locks the exact provider version — same idea as `requirements.txt` for Python.

```
terraform plan
```

This is a dry run. Terraform shows exactly what it will create without actually doing anything. I always run this before apply — no surprises.

```
terraform apply
```

Type `flask-key` for the key name, then `yes` to confirm.

Debugging — No Default Subnets

First error I hit:

```
Error: creating EC2 Instance: No subnets found for the default VPC
```

My AWS account had a default VPC but no default subnets inside it. Terraform couldn't place the EC2 anywhere.

Fix:

```
aws ec2 create-default-subnet --availability-zone ap-south-1a
```

Re-ran `terraform apply` and it worked.

Debugging — No Public IP

Apply succeeded but:

```
ec2_public_ip = ""
```

The EC2 was created without a public IP, so I couldn't reach it from the internet.

Fix: Added one line to the `aws_instance` block in `main.tf`:

```
associate_public_ip_address = true
```

Ran `terraform apply` again. This time Terraform destroyed the old EC2 and created a new one — because public IP association can't be changed on a running instance. That's fine. That's Terraform doing the right thing.

This time:

```
ec2_public_ip = "43.205.146.206"  
ssh_command = "ssh -i ~/.ssh/flask-key.pem ubuntu@43.205.146.206"
```

Phase 4 — Setting Up the EC2 Server

SSH into the freshly created EC2:

```
ssh -i ~/.ssh/flask-key.pem ubuntu@43.205.146.206
```

Installing Docker

```
sudo apt update && sudo apt upgrade -y  
sudo apt install docker.io docker-compose-v2 -y  
sudo systemctl start docker  
sudo systemctl enable docker  
sudo usermod -aG docker ubuntu  
newgrp docker
```

`systemctl enable` ensures Docker starts automatically on reboot. `usermod -aG docker ubuntu` adds ubuntu user to the docker group — otherwise every `docker` command needs `sudo`.

Installing Jenkins

Jenkins needs Java first:

```
sudo apt install openjdk-17-jdk -y
```

Then add the Jenkins repository and install:

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.ke
  gpg --dearmor | sudo tee /etc/apt/trusted.gpg.d/jenkins.gpg > /d

echo "deb [signed-by=/etc/apt/trusted.gpg.d/jenkins.gpg] \
  https://pkg.jenkins.io/debian-stable binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update --allow-insecure-repositories
sudo apt install jenkins -y --allow-unauthenticated
```

Honest note: The GPG key verification failed multiple times with various errors. I tried 4 different methods. Eventually I used `--allow-unauthenticated` to bypass it. For a production server I'd fix this properly — for a learning project on a temporary EC2, getting Jenkins installed was more important.

Give Jenkins Docker permissions — critical step:

```
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```

If you skip this, Jenkins will fail every build with "permission denied" when it tries to run `docker build`.

Adding Swap Memory — Important

t2.micro has 1GB RAM. Jenkins alone uses ~300MB. MySQL needs ~400MB. Flask needs ~100MB. That's already over 800MB on a 1GB machine.

The first time I ran the Jenkins pipeline, the EC2 completely froze. Couldn't SSH in, couldn't open Jenkins, nothing. The system ran out of memory and died.

The fix — swap space:

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
free -h
```

```
Mem:   954Mi   887Mi   72Mi
Swap:  1.4Gi   93Mi   1.3Gi
```

Swap is disk space used as overflow RAM. Slower than real RAM but prevents the system from freezing when memory gets tight. Adding it to `/etc/fstab` makes it survive reboots.

Jenkins Initial Setup

Get the initial admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Open `http://<EC2-IP>:8080` in browser, paste the password, click "Install suggested plugins", create an admin user.



Phase 5 — The Jenkins CI/CD Pipeline

The Jenkinsfile

This file lives in your repository and defines the pipeline. Jenkins reads it from GitHub on every build:

```
pipeline {
  agent any

  stages {
    stage('Clone Code') {
      steps {
        git branch: 'main', url: 'https://github.com/shlok
      }
    }

    stage('Build Docker Image') {
      steps {
        sh 'docker build -t flask-todo-app:latest .'
      }
    }

    stage('Deploy with Docker Compose') {
      steps {
        sh 'docker compose down || true'
        sh 'docker compose up -d --build'
      }
    }

    stage('Deployment Status') {
      steps {
        sh 'docker ps'
        echo 'Deployment successful! App running on port 5
      }
    }
  }
}
```

4 stages, clean and simple:

1. **Clone Code** — Jenkins pulls your latest GitHub code onto the EC2
2. **Build Docker Image** — builds a fresh Flask image from your Dockerfile
3. **Deploy with Docker Compose** — stops old containers, starts new ones
4. **Deployment Status** — runs `docker ps` to confirm everything is running, then prints success

The `|| true` on `docker compose down` means "if no containers are running, don't fail" — handles the first run where there's nothing to stop.

Creating the Pipeline in Jenkins

1. Dashboard → New Item → Pipeline → name it `flask-todo-pipeline`
2. Scroll to Pipeline section
3. Definition: **Pipeline script from SCM**
4. SCM: **Git**
5. Repository URL: your GitHub repo URL
6. Branch: `*/main`
7. Script Path: `Jenkinsfile`
8. Save

The Build That Took 51 Minutes to Fail

I clicked Build Now. Stage 1, 2, 3 went green. Stage 3 "Deploy with Docker Compose" started...

And kept going. 10 minutes. 20 minutes. 40 minutes. 51 minutes. Still running.

The EC2 froze again. Jenkins UI stopped responding.

This time it wasn't memory — it was **disk space**.

```
Usage of /: 99.8% of 6.71GB
```

The default EC2 root volume is 8GB. Docker had downloaded the MySQL image (~600MB), the Python image, build cache, Jenkins files — and the disk was completely full. Docker couldn't finish pulling images. Jenkins couldn't write logs. Everything froze.

Fix — free up disk first:

```
docker system prune -af
sudo rm -rf /var/lib/jenkins/workspace/flask-todo-pipeline
```

`docker system prune -af` removes all unused images, containers, and build cache. Freed 629MB instantly.

Fix — upgrade the disk via Terraform:

Added this to `main.tf`:

```
root_block_device {
  volume_size = 20
}
```

Ran `terraform apply`. Terraform expanded the volume to 20GB without destroying the EC2 — just modified the block device in place.

But AWS expanding the volume doesn't automatically tell the OS to use it. I had to do that manually:

```
sudo growpart /dev/xvda 1
sudo resize2fs /dev/root
df -h
```

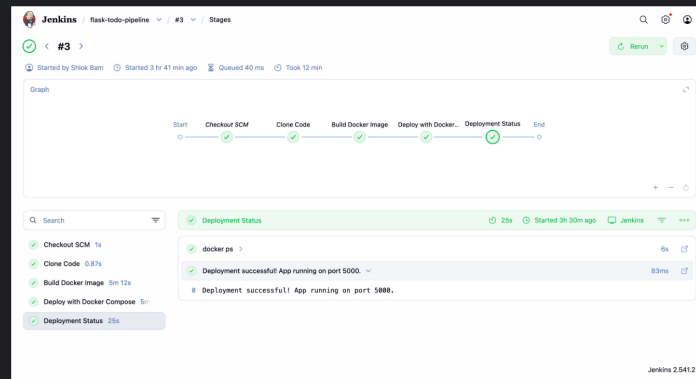
```
/dev/root 19G 6.2G 13G 34%
```

From 0% free to 13GB free. That's more like it.

```
ubuntu@ip-172-31-0-224:/var/lib/jenkins/workspace/flask-todo-pipeline$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        19G   7.0G   12G  38% /
tmpfs            478M   36K   478M   1% /dev/shm
tmpfs            191M   1.1M   190M   1% /run
tmpfs            5.0M    0   5.0M   0% /run/lock
/dev/xvda16      881M  162M   657M  20% /boot
/dev/xvda15      105M   6.2M   99M    6% /boot/efi
tmpfs            96M    16K   96M    1% /run/user/1000
```

Finally — All Green

Clicked Build Now again. This time with 13GB disk free and swap active:



```
✓ Clone Code - 0.87s
✓ Build Docker Image - 5 m 12s
✓ Deploy with Compose - 5m 48s
✓ Deployment Status - 25s
```

The Deploy stage took 5 minutes because it was downloading the MySQL image for the first time. Every build after that is much faster — the image is cached.

Phase 6 — The MySQL Connection Error

I opened `http://<EC2-IP>:5000` expecting to see my app.

Connection refused.

Checked the containers:

```
docker ps
```

```
flask-app  Restarting (1) 47 seconds ago
mysql      Up 4 minutes (healthy)
```

MySQL was healthy. Flask was crashing and restarting in a loop.

Checked Flask logs:

```
docker logs flask-app
```

```
mysql.connector.errors.DatabaseError: 1130 (HY000):
Host '172.18.0.3' is not allowed to connect to this MySQL server
```

```
host 172.18.0.3 is not allowed to connect to this MySQL server
```

This one took me a while to understand.

MySQL 8.0 by default only allows the root user to connect from `localhost`. But Flask is running in a separate container with IP `172.18.0.3`. From MySQL's perspective, that's a remote host — and root isn't allowed from remote hosts.

Fix:

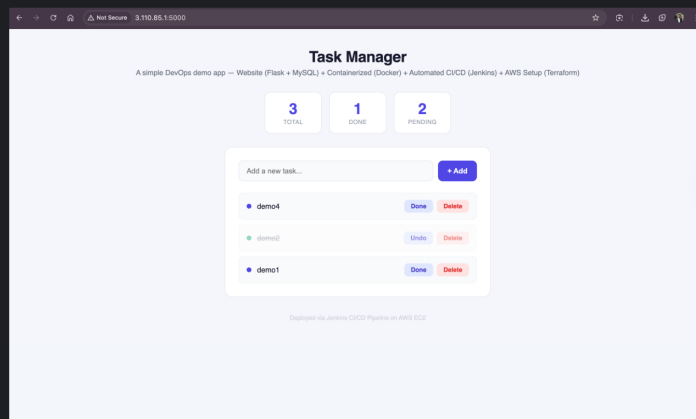
```
docker exec -it mysql mysql -uroot -proot -e \  
"GRANT ALL PRIVILEGES ON *.* TO 'root'@'%' IDENTIFIED BY 'root';
```

`'root'@'%'` means "allow root user to connect from any host". The `%` is a wildcard.

Then:

```
docker compose down  
docker compose up -d --build  
docker ps
```

Both containers running healthy. Opened `http://<EC2-IP>:5000`.



It worked. The app was live on AWS.

Phase 7 — GitHub Webhook Automation

The pipeline works. But right now I have to click "Build Now" manually every time I push code. That defeats the purpose of CI/CD.

Webhooks fix this.

What is a Webhook?

A webhook is basically GitHub saying "hey Jenkins, someone just pushed code" — it sends an HTTP POST request to Jenkins every time a push happens. Jenkins receives it and automatically starts the pipeline.

Setting It Up

In GitHub:

1. Repository → Settings → Webhooks → Add webhook

2. Payload URL: `http://<EC2-IP>:8080/github-webhook/`

3. Content type: `application/json`

4. Events: "Just the push event"

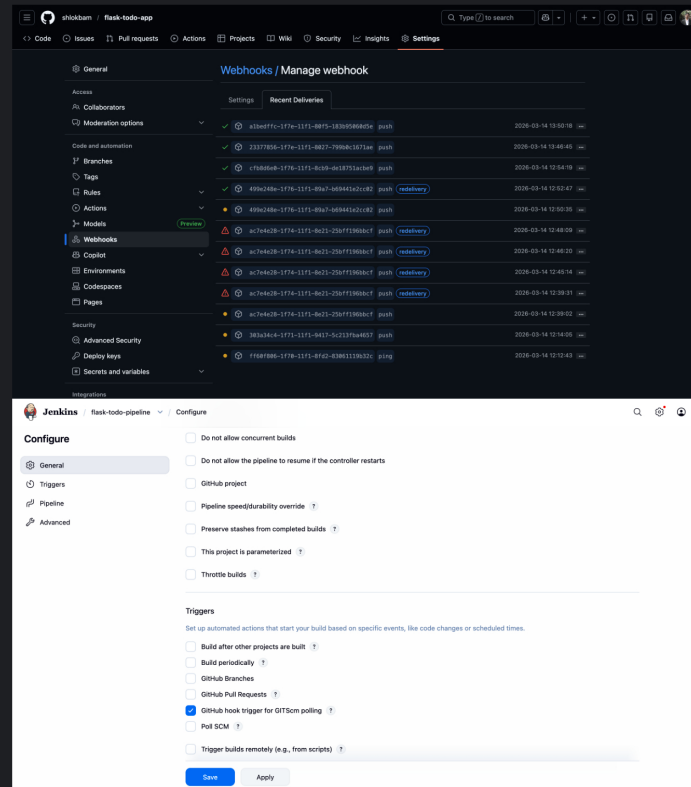
5. Save

In Jenkins:

1. Pipeline → Configure

2. Build Triggers → check "GitHub hook trigger for GITScm polling"

3. Save



The screenshot shows two parts of the Jenkins interface. The top part is the 'Webhooks / Manage webhook' page for the 'flask-todo-app'. It lists several webhooks with their URLs, events, and delivery status. The bottom part is the 'Configure' page for the 'flask-todo-pipeline'. Under the 'Triggers' section, the 'GitHub hook trigger for GITScm polling' checkbox is checked, and the 'Save' button is highlighted.

Testing It

Made a small change — updated the footer text in `index.html` :

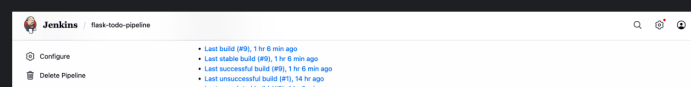
```
<!-- changed from -->
<Footer>Deployed via Jenkins CI/CD Pipeline on AWS EC2</Footer>

<!-- changed to -->
<Footer>Auto-deployed via Jenkins CI/CD | Flask + Docker + AWS</fo
```

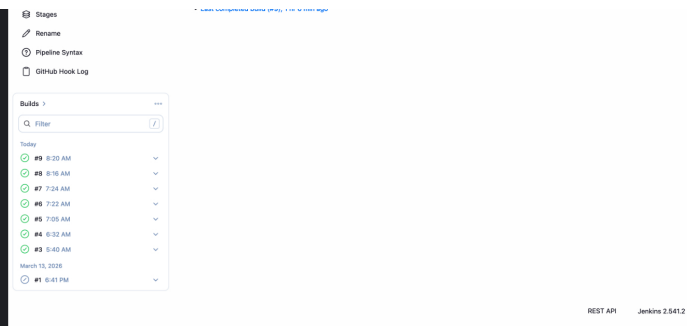
Committed and pushed:

```
git add .
git commit -m "update footer text"
git push origin main
```

Within seconds, Jenkins dashboard showed a new build starting automatically. No clicking. The pipeline ran all 4 stages and deployed. Refreshed the website — footer was updated.



The screenshot shows the Jenkins 'Pipeline Run' page for the 'flask-todo-pipeline'. It displays a list of recent builds with their status and timestamps. The 'Last successful build' is highlighted, indicating a successful deployment.



That moment — seeing your code go from your laptop to a live server automatically — is genuinely satisfying. That's CI/CD working exactly as intended.

Everything That Went Wrong — Summary

Here's every problem I hit and how I fixed it, for quick reference:

Problem	Cause	Fix
Port 3306 already in use	Local MySQL using the port	Changed to <code>3307:3306</code> in <code>docker-compose.yml</code>
No subnets found	New AWS account without default subnets	<code>aws ec2 create-default-subnet --availability-zone ap-south-1a</code>
No public IP on EC2	Missing <code>associate_public_ip_address = true</code> in Terraform	Added the line, re-applied
EC2 froze completely	t2.micro ran out of 1GB RAM	Added 2GB swap space
Jenkins GPG key error	Key format incompatible with Ubuntu 24.04	Used <code>--allow-unauthenticated</code> flag
Jenkins startup timeout	Default 90s timeout too short for t2.micro	Increased to 300s via <code>systemd</code> override
Disk full, pipeline stuck	8GB default volume filled by Docker images	Upgraded to 20GB via Terraform, expanded filesystem
Flask can't connect to MySQL	MySQL 8.0 restricts root to localhost	<code>GRANT ALL PRIVILEGES TO 'root'@'%'</code>

Every single one of these errors taught me something. The disk space issue taught me about Docker layer caching. The MySQL permissions error taught me about MySQL's default security model. The RAM issue taught me about swap memory.

You learn more from things breaking than from things working.

What I'd Do Differently

1. Use t2.medium instead of t2.micro t2.micro with 1GB RAM is genuinely painful for running Jenkins + Docker + MySQL. It works, but with swap memory and timeouts. 2GB RAM makes everything smoother.

2. Use environment variables for secrets The MySQL password is hardcoded as "root" in `docker-compose.yml`. In a real project I'd use AWS Secrets

Manager or at minimum a `.env` file that's never committed to GitHub.

3. Add a `terraform.tfvars` file Instead of typing `flask-key` every time Terraform asks, I'd store it in a `terraform.tfvars` file:

```
key_name = "flask-key"
```

4. Use `user_data` in Terraform Terraform's `user_data` lets you run a shell script when EC2 first starts — so Docker and Jenkins get installed automatically as part of `terraform apply`. No manual SSH setup needed.

Key Takeaways

Docker networking — containers communicate by service name, not localhost. This is one of those things that sounds obvious in theory and confuses everyone in practice.

Infrastructure as Code — once you understand Terraform, you'll never want to click through AWS console again. The ability to `terraform destroy` and `terraform apply` and get back exactly what you had is genuinely powerful.

CI/CD is just automation — it sounds complex but it's literally: code change → trigger → build → deploy. The magic is that each step is reliable and repeatable.

Real projects break — every tutorial shows you the happy path. Real projects hit disk limits, memory constraints, GPG key incompatibilities, and MySQL permission errors. Debugging these is the actual job.

Resources

- GitHub repo: github.com/shlokbam/flask-todo-app
- [Terraform AWS provider docs](#)
- [Jenkins Pipeline syntax](#)
- [Docker Compose reference](#)

If you built this or hit different errors, share in the comments. Would love to know what broke for you.

👁 30 views

Comments

💬 Join the discussion

No comments yet. [Be the first to comment.](#)

More from this blog

I Built an AI-Powered Mock Interview Platform from Scratch — Here's Everything That Went Wrong

A full walkthrough of building MockVue — React + FastAPI + TiDB Cloud + Groq AI + face-api.js — including every bug, every architectural decision, and every "why is this not working" mome...

Apr 6, 2026 · 28 min read · 32



I Built an AI Data Analyst App from Scratch — Here's How I Taught a Flask App to Think

A full walkthrough of building DataLens — CSV uploads, Groq/Llama AI insights, auto-generated charts, user auth, persistent chat history, and PDF export. Including every bug, ever...

Mar 26, 2026 · 13 min read · 10



Build. Break. Ship.

3 posts



© 2026 Build. Break. Ship.

[Archive](#) [Privacy](#) [Terms](#)

[Sitemap](#) [RSS](#)

 hashnode